PRACTICAL NO 2: PERFORM IMAGE STITCHING

```
import cv2
import numpy as np

# Load images
image1 = cv2.imread("D:\CV\pex.jpg")
image2 = cv2.imread("D:\CV\pex1.jpg")
print("Image 1 shape:", image1.shape)
print("Image 2 shape:", image2.shape)

# Convert images to grayscale
gray1 = cv2.cvtColor(image1, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(image2, cv2.COLOR_BGR2GRAY)

# Detect keypoints and compute descriptors
sift = cv2.SIFT_create()
keypoints1, descriptors1 = sift.detectAndCompute(gray1, None)
keypoints2, descriptors2 = sift.detectAndCompute(gray2, None)

# Match descriptors between the two images
matcher = cv2.BFMatcher()
matches = matcher.match(descriptors1, descriptors2)

# Sort matches by distance
matches = sorted(matches, key=lambda x: x.distance)

# Extract matched keypoints
points1 = np.float32([keypoints1[match.queryIdx].pt for match in matches]).reshape(-1, 1, 2)
print("Number of points in points1:", len(points1))
```

```
points2 = np.float32([keypoints2[match.trainIdx].pt for match in matches]).reshape(-1, 1, 2)
print("Number of points in points2:", len(points2))

# Find homography matrix
homography, _ = cv2.findHomography(points1, points2, cv2.RANSAC)

# Warp image1 to align with image2
height, width = gray2.shape
stitched_image = cv2.warpPerspective(image1, homography, (width, height))

# Combine the stitched image with image2
stitched_image[0:image2.shape[0], 0:image2.shape[1]] = image2

# Display the stitched image
cv2.imshow('Stitched Image', stitched_image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output:

